# 山东大学 计算机科学与技术 学院

# 数据结构与算法 课程实验报告

| 学号：202200130077 | 姓名：薛润泽 | 班级：人工智能 |
|---|---|---|
| 实验题目：实验四 链式描述线性表 | | |
| 实验学时：2 | 实验日期： 2023.9.27 | |

**实验目的：**

1. 掌握线性表结构、链式描述方法（链式存储结构）、链表的实现。
2. 掌握链表迭代器的实现与应用。

**软件开发工具：**

DEV C++

## 1. 实验内容

题目描述：

（1）要求封装链表类，链表迭代器类；

（2）链表类需提供操作：在指定位置插入元素，删除指定元素，搜索链表中是否有指（3）定元素，原地逆置链表，输出链表；

（4）不得使用与链表实现相关的 STL。

操作要求：实现插入、删除、逆置、查询、输出等操作

## 2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

数据结构与算法：链表，指针，迭代器

### 实验 A

思路描述:

(1) 插入操作:chaininsert() 对插入位置进行判断，如果插入时还没有节点，则将插入节点作为头节点，如果插入位置索引为 0,那么将插入节点指向头节点后，再将该插入节点作为头节点。其他情况下，找到插入位置的前一个节点，并将该节点指向插入节点，将插入节点指向原位置节点。

(2) 删除操作:chaindelete()先遍历链表，分别用指针记录遍历到现在的节点和上一个节点，遍历到要删除的节点后，将该节点的上一个节点的 next 指针指到该节点的下一个节点，再 delete 本节点。如果遍历到最后也没有找到要删除的节点，那么输出-1。特别的，如果是删除的头结点，在删除完结点后，要将头结点指针的值改变为下一个节点指针的值。

(3) 逆置操作:chaininverse()定义两个指针，分别记录操作节点 cntNode 和操作节点的左节点 leftptr，并将 cntNode 初始化为 firstNode，如果 cntNode 不是空指针，那么执行逆置操作，新定义一个指针 tempright，赋值为 cntNode->next，然后将 cntNode->next 赋值为 leftptr，使该操作节点指向上一个节点，再将上一个节点跳转到操作节点，将操作节点跳转到下一个节点，即 tempright 节点。

(4) 查询操作:chainfind()，遍历链表，如果找到该元素则输出索引，如果没有则输出-1

(5) 计算索引与元素异或和:使用迭代器遍历链表，重载++,*等运算符，使迭代器具有自加和解引用等功能，在遍历时完成异或和计算。

(6) 迭代器类 将迭代器类 iterator 作为 Chain 链表类的一个内置类，重载++，*()运算符，实现迭代器的自加和解引用。并写一个 isend()函数，判断是否到达尾端。使用迭代器类的时候要首先使用 chain 类的 getfirstNode，向 iterator 的构造函数提供链表头指针。

## 实验 B

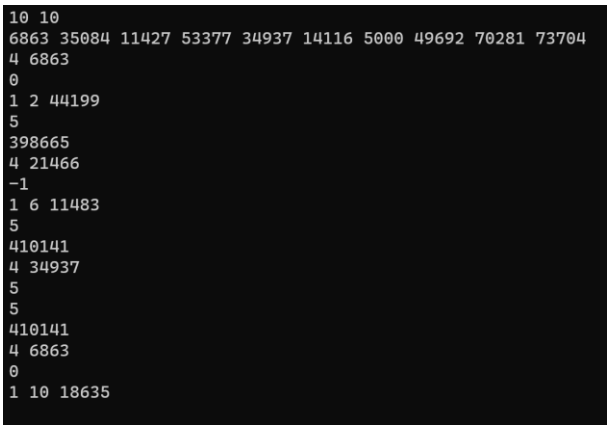
思路描述：

(1) 排序操作:chainsort(),使用冒泡排序,从链表头结点开始遍历,如果该节点与下一个节点的值相比要更大，那么交换这两个节点的值。

(2) 合并操作:因为两个要合并的链表已经是有序的，所以在合并的过程中依次比较两个链表的当前节点值大小，将更小的那一个合并到新链表中，并且令迭代器自加指到下一个节点。重复该操作过程。直到有一个链表已经遍历到最后，那么将另一个链表的所有值全部合并到新链表。

3. 测试结果（测试输入，测试输出）

## 实验 A




## 实验 B




4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

结果与预期相符。

在 debug 的过程中可以发现，在某一个操作中代码执行有误，可能不是该操作本身出现了问题，而是其他操作出现了逻辑的不严密甚至是错误。所以在检查完一个操作的正确性后，可以放眼整个代码，思考什么样的操作也会造成本次错误，然后去可能执行该操作的地方进一步检查。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

## //实验 A

```
#include<iostream>

class Node{      //Node 作为链表结点
public:
```

```
    Node(int val):val(val)
    {
        next = nullptr;
    };
    Node *next;
    ~Node(){};
    int val;
};

class Chain{    //链表
public:
    Chain(){
    firstNode = nullptr;
    listnum = 0;
    };
    Node* getfirstNode(){
        return firstNode;
    }
    void createChain(int n);//实现开始时生成一个链表
    void chaininsert();     //实现链表插入操作
    void chaindelete();     //实现链表删除操作
    void chaininverse();    //实现链表逆置操作
    void chainfind();       //实现链表查询操作
    void exclusiveor(Node*first);//输出异或和
    Node* begin();//返回 firstNode
    ~Chain(){
        while(firstNode != nullptr){
            Node *temp = firstNode->next;
            delete firstNode;
            firstNode = temp;
        }
    }
    class iterator{         //迭代器类，重载++，*等运算符，实现指针功能
    public:
        iterator(Node* temp):temp(temp){};
        Node* temp;
        bool isend(){
            if(temp==nullptr) return 1;
            else return 0;
        }
        iterator &operator++(int){
            this->temp = temp->next;
            return *this;
        }
        int operator*(){
            return temp->val;
```

```
        }
    };


private:
    Node* firstNode;
    int listnum = 0 ;//记录结点个数
};

void Chain::createChain(int n){
    listnum = n;
    Node* tempptr = nullptr;
    for(int i = 0; i < n; i++){
        int val;
        std::cin>>val;
        Node* temp =  new Node(val);
        if(i == 0){             //如果链表中还没有结点，直接把该结点作为头结点
            firstNode = temp;
            tempptr = temp;
        }
        else {
        tempptr->next = temp;
        tempptr = tempptr->next;
        }
    }
}
void Chain::chaininsert(){
    listnum++;          //记录结点个数++
    int indexnum;       //输入 idx
    std::cin>>indexnum;
    int val;
    std::cin>>val;
    Node* element = new Node(val);//new 一个新结点
    if(firstNode ==nullptr){//如果还没有结点
        firstNode = element;
    }
    else if(indexnum == 0){                 //如果插入结点处为链表第一个位置
        element->next = firstNode;
        firstNode = element;
    }
    else {
        Node *leftptr = firstNode;
        for(int i = 0; i < indexnum - 1; i++){
            leftptr = leftptr->next;
        }
        element->next = leftptr->next;
```

```
        leftptr->next = element;
    }
    return ;
}
void Chain::chaindelete(){
    listnum --;
    int val;
    std::cin>>val;          //输入要删除的值
    Node* tempptr = firstNode;//记录遍历到的现在的结点
    Node* lastptr = firstNode;//记录现在的结点的上一个结点
    while(tempptr != nullptr){
        if(tempptr->val == val){
            if(tempptr == firstNode){
                lastptr->next = tempptr->next;
                firstNode = tempptr->next;
                tempptr->next = nullptr;
                delete tempptr;
                return;
            }
            else{
                lastptr->next = tempptr->next;
                tempptr->next = nullptr;
                delete tempptr;
                return;
            }
        }
        lastptr = tempptr;
        tempptr = tempptr->next;
    }
    std::cout<<"-1"<<'\n';
    return ;
}
void Chain::chaininverse(){
    Node *leftptr = nullptr;    //记录操作结点的左节点
    Node *cntNode = firstNode;  //记录操作结点
    if(cntNode == nullptr) return;//如果该操作结点已经是空，那么说明已经完成了逆置
    while(cntNode!= nullptr){
        Node *tempright = cntNode->next;    //记录操作结点的右结点
        cntNode->next = leftptr;            //将现在结点指向记录的左节点
        leftptr = cntNode;                  //将左节点跳转到现在结点
        cntNode = tempright;                //将现在结点跳转到右结点
    }
    firstNode = leftptr;                    //因为 cntNode == nullptr, 所以 leftptr 是最后
的结点，将头指针单独指向 leftptr 完成逆置
}
void Chain::chainfind(){
```

```
    int val;
    std::cin>>val;
    Node* tempptr =  firstNode;
    int count = 0;
    while(tempptr!=nullptr){
        if(tempptr->val == val){
            std::cout<<count<<'\n';
            return ;
        }
        tempptr = tempptr->next;
        count++;
    }
    std::cout<<"-1"<<'\n';
}
int main(){
    int N, Q;
    std::cin>>N>>Q;
    Chain chain;
    chain.createChain(N);
    while(Q > 0){
        Q--;
        int sym;
        std::cin>>sym;
        if(sym ==1){
            chain.chaininsert();
        }
        else if(sym == 2){
            chain.chaindelete();
        }
        else if(sym == 3){
            chain.chaininverse();
        }
        else if(sym ==4){
            chain.chainfind();
        }
        else if(sym == 5){
            Chain::iterator it(chain.getfirstNode());
            int sumor = 0; int countnum = 0;
            int count = 0;
            for(; !it.isend(); it++)
            {
                sumor+= countnum^ *(it);
                countnum++ ;
            }
            std::cout<<sumor<<'\n';
        }
```

```
        else{
            std::cout<<"wrong number"<<'\n';
            return 0;
        }
    }
    return 0;
}
```

## //实验 B

```
#include<iostream>

class Node{
public:

    Node(int val):val(val)
    {
        next = nullptr;
    };
    Node *next;
    ~Node(){};
    int val;
};

class Chain{
public:
    Chain(){
    firstNode = nullptr;
    listnum = 0;
    };
    Node* getfirstNode(){
        return firstNode;
    }
    void createChain(int n);//实现开始时生成一个链表
    void chaininsert();     //实现链表插入操作
    void chaindelete();     //实现链表删除操作
    void chaininverse();    //实现链表逆置操作
    void chainfind();       //实现链表查询操作
    void exclusiveor(Node*first);//输出异或和
    void chainsort();       //实现链表排序
    void merge(Chain &A, Chain &B);//实现链表合并，保留原链表的同时生成一个新链表
    Node* begin();      //返回 firstNode
    ~Chain(){
        while(firstNode != nullptr){
            Node *temp = firstNode->next;
            delete firstNode;
            firstNode = temp;
```

```
        }
    }
    class iterator{     //迭代器类，重载++，*等运算符，实现指针功能
    public:
        iterator(Node* temp):temp(temp){};
        Node* temp;
        bool isend(){
            if(temp==nullptr) return 1;
            else return 0;
        }
        iterator &operator++(int){
            this->temp = temp->next;
            return *this;
        }
        Node* getptr(){
            return temp;
        }
        int operator*(){
            return temp->val;
        }
    };


private:
    Node* firstNode;
    int listnum = 0 ;//记录结点个数
};

void Chain::createChain(int n){
    listnum = n;
    Node* tempptr = nullptr;
    for(int i = 0; i < n; i++){
        int val;
        std::cin>>val;
        Node* temp =  new Node(val);
        if(i == 0){//如果链表中还没有结点，直接把该结点作为头结点
            firstNode = temp;
            tempptr = temp;
        }
        else {
        tempptr->next = temp;
        tempptr = tempptr->next;
        }
    }
}
void Chain::chaininsert(){
```

```
    listnum++;          //记录结点个数++
    int indexnum;       //输入 idx
    std::cin>>indexnum;
    int val;
    std::cin>>val;
    Node* element = new Node(val);//new 一个新结点
    if(firstNode ==nullptr){//如果还没有结点
        firstNode = element;
    }
    else if(indexnum == 0){                 //如果插入结点处为链表第一个位置
        element->next = firstNode;
        firstNode = element;
    }
    else {
        Node *leftptr = firstNode;
        for(int i = 0; i < indexnum - 1; i++){
            leftptr = leftptr->next;
        }
        element->next = leftptr->next;
        leftptr->next = element;
    }
    return ;
}
void Chain::chaindelete(){
    listnum --;
    int val;
    std::cin>>val;            //输入要删除的值
    Node* tempptr = firstNode;//记录遍历到的现在的结点
    Node* lastptr = firstNode;//记录现在的结点的上一个结点
    while(tempptr != nullptr){
        if(tempptr->val == val){
            lastptr->next = tempptr->next;
            tempptr->next = nullptr;
            delete tempptr;

            return;
        }
        lastptr = tempptr;
        tempptr = tempptr->next;
    }
    std::cout<<"-1"<<'\n';
    return ;
}
void Chain::chaininverse(){
    Node *leftptr = nullptr; //记录操作结点的左节点
    Node *cntNode = firstNode;  //记录操作结点
```

```
    if(cntNode == nullptr) return;//如果该操作结点已经是空，那么说明已经完成了逆置
    while(cntNode!= nullptr){
        Node *tempright = cntNode->next;//记录操作结点的右结点
        cntNode->next = leftptr;        //将现在结点指向记录的左节点
        leftptr = cntNode;              //将左节点跳转到现在结点
        cntNode = tempright;            //将现在结点跳转到右结点
    }
    firstNode = leftptr;                //因为 cntNode == nullptr, 所以 leftptr 是最后的结点，将头指针单独指向 leftptr 完成逆置
}

void Chain::chainfind(){
    int val;
    std::cin>>val;
    Node* tempptr =  firstNode;
    int count = 0;
    while(tempptr!=nullptr){
        if(tempptr->val == val){
            std::cout<<count<<'\n';
            return ;
        }
        tempptr = tempptr->next;
        count++;
    }
    std::cout<<"-1"<<'\n';
}

Node* Chain::begin(){
    return firstNode;
}
void swapnode(Node*a, Node*b){
    int temp = a->val;
    a->val = b->val;
    b->val = temp;
}
void Chain::chainsort(){
    for(int i = 0; i < listnum; i++){// 使用冒泡排序，如果该节点值比下一个节点值大，那么交换两个节点值
        Node* temp = firstNode;
        while(temp!=nullptr){
            if(temp->next == nullptr) break;
            if(temp->val > (temp->next)->val) swapnode(temp, temp->next);
            temp = temp->next;
        }
    }
}
```

```
void Chain::merge(Chain&A, Chain&B){    //合并两个链表，在保留两个原链表的同时生成新链表
    Chain::iterator itA(A.getfirstNode());
    Chain::iterator itB(B.getfirstNode());
    Node* temp = nullptr;
    firstNode = nullptr;
    for(; !itA.isend();itA++){          //因为合并的两个链表有序，所以在合并链表的时候比较两个节点值
        int sym = 0 ;                   //终断判断标志
        for(; !itB.isend();itB++){
            if(firstNode == nullptr){
                if(*itA > *itB){
                    firstNode = new Node(*itB);
                    temp = firstNode;
                    }
                else sym = 1;//让外层 itA++
                }
            else{
                if(*itA >*itB){
                    temp->next = new Node(*itB);
                    temp = temp->next;
                }
                else sym = 1;
            }
            if(sym == 1)break;
        }
        if(!itA.isend()){
            if(firstNode == nullptr){
                firstNode = new Node(*itA);
                temp = firstNode;
            }
            else{
            temp->next = new Node(*itA);
            temp = temp->next;
            }
        }
    }
    for(; !itB.isend();itB++){              //如果当 A 的链表结束时，B 链表中还有值，那么把 B 链表中剩下的那些添加入新链表中
        if(firstNode == nullptr){
                firstNode = new Node(*itB);
                temp = firstNode;}
        else{
                temp->next = new Node(*itB);
                temp = temp->next;  }
    }
}
```

```
int main(){
    Chain chain1;
    Chain chain2;
    int N, M;
    std::cin>>N;
    std::cin>>M;
    chain1.createChain(N);
    chain2.createChain(M);
    chain1.chainsort();
    chain2.chainsort();
    Chain chain3;
    chain3.merge(chain1,chain2);    //合并链表
    Chain::iterator it1(chain1.getfirstNode());
    Chain::iterator it2(chain2.getfirstNode());
    Chain::iterator it3(chain3.getfirstNode());
    int ans1 = 0,count1 = 0,ans2 = 0, count2 = 0, ans3 = 0, count3 = 0;
    for(;!it1.isend();it1++){       //依次输出三个要求的异或和
        ans1 += *it1^count1;
        count1++;
    }
    for(;!it2.isend();it2++){
        ans2 += *it2^count2;
        count2++;
    }
    for(;!it3.isend();it3++){
        ans3 += *it3^count3;
        count3++;
    }
    std::cout<<ans1<<'\n'<<ans2<<'\n'<<ans3<<'\n';
    return 0;
}
```